

1. Basic Calculator

Code:

student:-

```
write('Name:- Magdelin Stefey'), nl,  
write('Register Number:- 192511329'), nl.
```

add(A,B,C) :-

C is A+B.

subtract(A,B,C) :-

C is A-B.

multiply(A,B,C) :-

C is A*B.

divide(A,B,C) :-

C is A/B.

Output:



The screenshot shows a Prolog IDE with a code editor on the left and an output window on the right. The code editor contains the following Prolog code:

```
1 student :-  
2   write('Name:- Magdelin Stefey'), nl,  
3   write('Register Number:- 192511329'), nl.  
4  
5 add(A,B,C) :-  
6   C is A+B.  
7  
8 subtract(A,B,C) :-  
9   C is A-B.  
10  
11 multiply(A,B,C) :-  
12   C is A*B.  
13  
14 divide(A,B,C) :-  
15   C is A/B.
```

The output window shows the results of the queries:

```
student.  
Name:- Magdelin Stefey  
Register Number:- 192511329  
true  
  
?- add(10,5,C).  
C = 15  
true  
  
?- subtract(10,5,C).  
C = 5  
true  
  
?- multiply(10,5,C).  
C = 50  
true  
  
?- divide(10,5,C).  
C = 2  
true
```

At the bottom of the output window, there are buttons for 'Examples', 'History', 'Solutions', 'table results', and a 'Run!' button.

2. All Comparison

Code :

student:-

```
write('Name:- Magdelin Stefey'), nl,
```

```
write('Register Number:- 192511329'), nl.
```

greater(A,B) :-

A > B.

less(A,B) :-

A < B.

greater_equal(A,B) :-

A >= B.

less_equal(A,B) :-

A <= B.

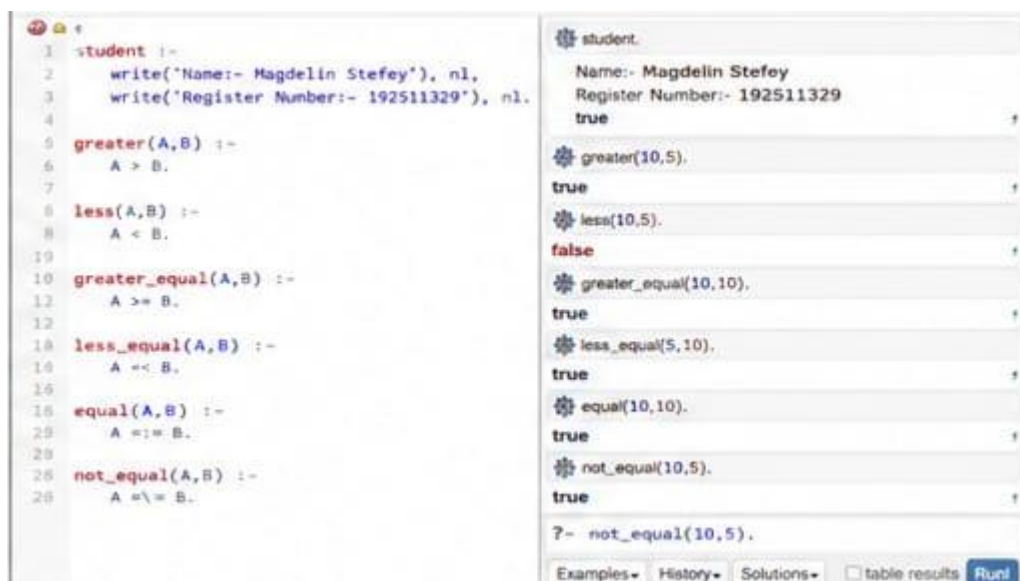
equal(A,B) :-

A == B.

not_equal(A,B) :-

A \= B.

Output:



The screenshot shows a Prolog IDE with two panes. The left pane contains the following code:

```
1 student :-
2   write('Name:- Magdelin Stefey'), nl,
3   write('Register Number:- 192511329'), nl.
4
5 greater(A,B) :-
6   A > B.
7
8 less(A,B) :-
9   A < B.
10
11 greater_equal(A,B) :-
12   A >= B.
13
14 less_equal(A,B) :-
15   A <= B.
16
17 equal(A,B) :-
18   A == B.
19
20 not_equal(A,B) :-
21   A \= B.
```

The right pane shows the output of the code:

```
student.
Name:- Magdelin Stefey
Register Number:- 192511329
true
?- greater(10,5).
true
?- less(10,5).
false
?- greater_equal(10,10).
true
?- less_equal(5,10).
true
?- equal(10,10).
true
?- not_equal(10,5).
true
?- not_equal(10,5).
```

3.Vote eligibility

Code:

vote :-

```
write('Name:- Magdelin Stefey'), nl,
```

```
write('Register Number:- 192511329'), nl.
```

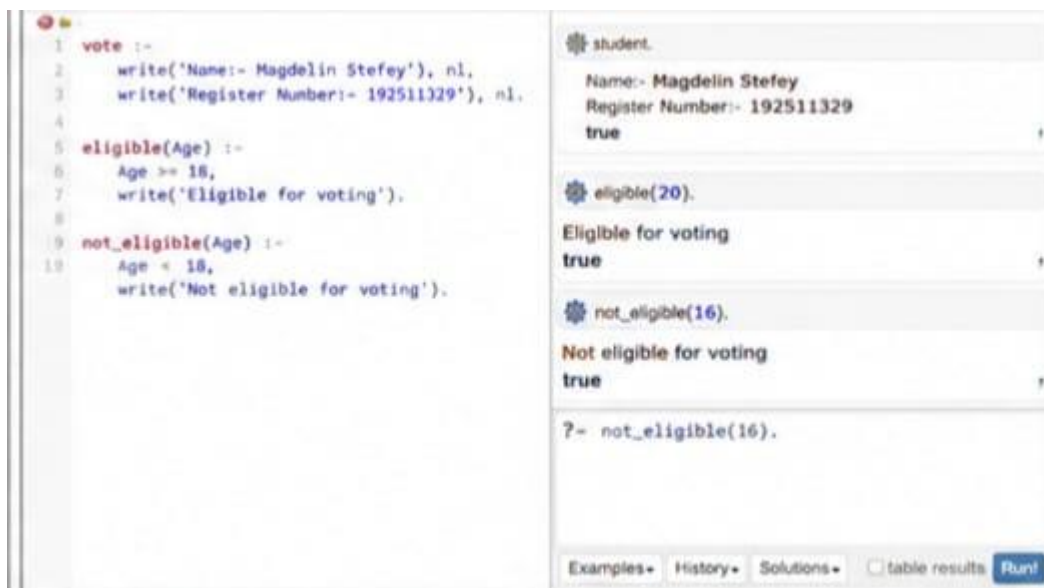
eligible(Age) :-

```
Age >= 18,  
write('Eligible for voting').
```

not_eligible(Age) :-

```
Age < 18,  
write('Not eligible for voting').
```

Output:



The screenshot shows a Prolog IDE with a code editor on the left and a console on the right. The code editor contains the following Prolog code:

```
1 vote :-  
2   write('Name:- Magdelin Stefey'), nl,  
3   write('Register Number:- 192511329'), nl.  
4  
5 eligible(Age) :-  
6   Age >= 18,  
7   write('Eligible for voting').  
8  
9 not_eligible(Age) :-  
10  Age < 18,  
11  write('Not eligible for voting').
```

The console on the right shows the execution of the code:

```
student.  
Name:- Magdelin Stefey  
Register Number:- 192511329  
true  
  
eligible(20).  
Eligible for voting  
true  
  
not_eligible(16).  
Not eligible for voting  
true  
  
?- not_eligible(16).
```

At the bottom of the console, there are buttons for "Examples", "History", "Solutions", a checkbox for "table results", and a "Run" button.

4.celsius_to_fahrenheit

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,  
write('Register Number:- 192511329'), nl.
```

convert(C,F) :-

F is (C * 9/5) + 32.

Output:

```

1 student :-
2     write('Name:- Magdelin Stefey'), nl,
3     write('Register Number:- 192511329'), nl.
4
5 convert(C,F) :-
6     F is (C * 9/5) + 32.

```

student.

Name:- Magdelin Stefey
Register Number:- 192511329
true

convert(25,F).

F = 77

?- convert(25,F).

Examples History Solutions ☐ table results Run!

5.Pass/Fail 3 sub

Code:

student :-

```

write('Name:- Magdelin Stefey'), nl,
write('Register Number:- 192511329'), nl.

```

result(M1,M2,M3) :-

```

(M1 >= 40, M2 >= 40, M3 >= 40 ->
    write('Pass')
;
    write('Fail')
).

```

Output:

```

1 student :-
2     write('Name:- Magdelin Stefey'), nl,
3     write('Register Number:- 192511329'), nl.
4
5 result(M1,M2,M3) :-
6     (M1 >= 40, M2 >= 40, M3 >= 40 ->
7         write('Pass')
8     ;
9         write('Fail')
10    ).

```

student.

Name:- Magdelin Stefey
Register Number:- 192511329
true

result(75,80,65).

Pass
true

result(75,30,65).

Fail
true

?- result(75,30,65).

Examples History Solutions ☐ table results Run!

6. Avg of 5 numbers

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,  
write('Register Number:- 192511329'), nl.
```

average(A,B,C,D,E,Avg) :-

Avg is (A+B+C+D+E)/5.

Output:



The screenshot shows a Prolog IDE with two panels. The left panel contains the following code:

```
1 student :-  
2   write('Name:- Magdelin Stefey'), nl,  
3   write('Register Number:- 192511329'), nl.  
4  
5 average(A,B,C,D,E,Avg) :-  
6   Avg is (A+B+C+D+E)/5.
```

The right panel shows the execution results:

```
student.  
Name:- Magdelin Stefey  
Register Number:- 192511329  
true  
  
average(10,20,30,40,50,Avg).  
Avg = 30  
  
?- average(10,20,30,40,50,Avg).
```

At the bottom of the right panel, there are buttons for 'Examples', 'History', 'Solutions', a checkbox for 'table results', and a 'Run!' button.

7. Multiplication Table

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,  
write('Register Number:- 192511329'), nl.
```

table(N) :-

table(N, 1).

table(_, 11) :- !.

table(N, I) :-

R is N * I,

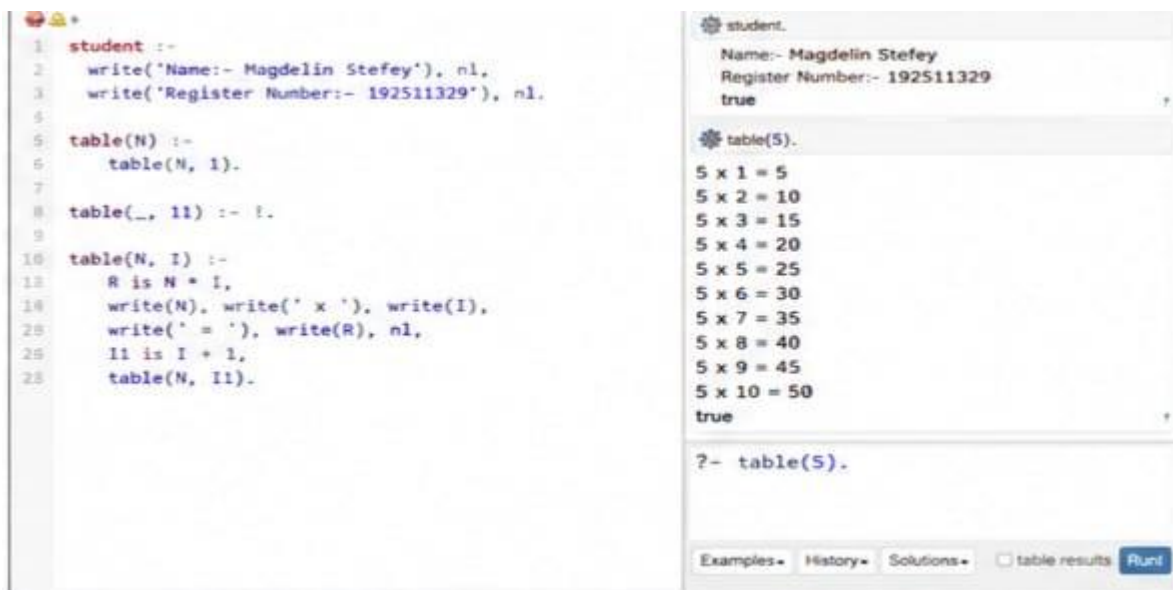
write(N), write(' x '), write(I),

write(' = '), write(R), nl,

I1 is I + 1,

table(N, I1).

Output:



The screenshot shows a Prolog IDE with two panes. The left pane contains the following code:

```
1 student :-
2   write('Name:- Magdelin Stefey'), nl,
3   write('Register Number:- 192511329'), nl.
4
5 table(N) :-
6   table(N, 1).
7
8 table(_, I1) :- !.
9
10 table(N, I) :-
11   R is N * I,
12   write(N), write(' x '), write(I),
13   write(' = '), write(R), nl,
14   I1 is I + 1,
15   table(N, I1).
```

The right pane shows the output of the code. It starts with the student information:

```
student.
Name:- Magdelin Stefey
Register Number:- 192511329
true
```

Then it shows the results of the `table(5).` query:

```
table(5).
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
true
```

At the bottom, there is a prompt `?- table(5).` and buttons for Examples, History, Solutions, table results, and Run.

8.Student teacher subject relation

Code:

student :-

write('Name:- Magdelin Stefey'), nl,

write('Register Number:- 192511329'), nl.

teacher(teaches, maths, ravi).

teacher(teaches, science, priya).

teacher(teaches, english, kumar).

```

student_subject(Magdelin Stefey, maths).
student_subject(Magdelin Stefey, science).
student_subject(Magdelin Stefey, english).

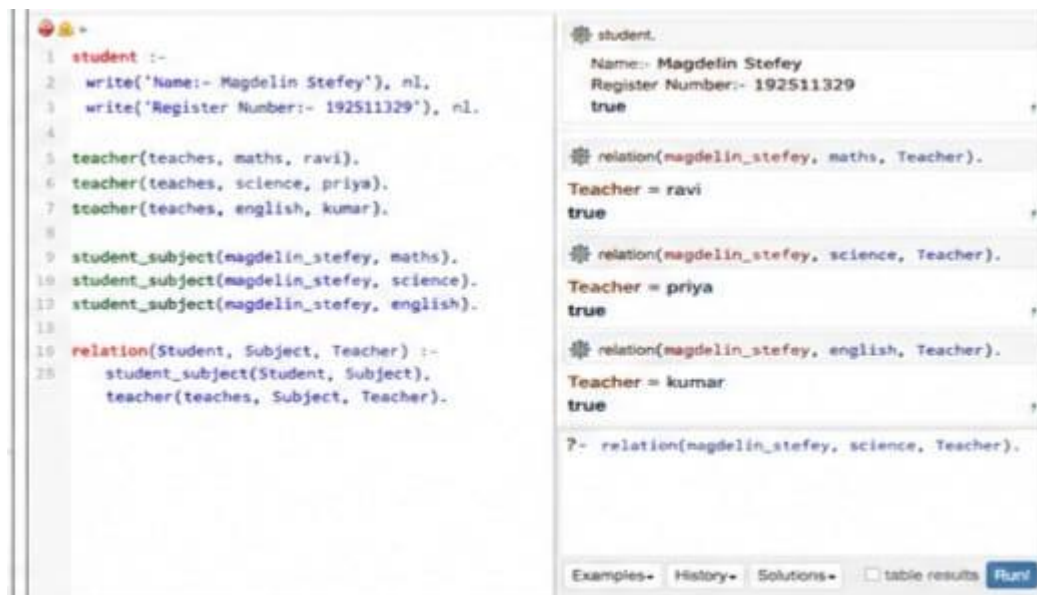
```

```

relation(Student, Subject, Teacher) :-
    student_subject(Student, Subject),
    teacher(teaches, Subject, Teacher).

```

Output:



The screenshot shows a Prolog IDE with two panes. The left pane contains the following code:

```

1 student :-
2   write('Name:- Magdelin Stefey'), nl,
3   write('Register Number:- 192511329'), nl.
4
5 teacher(teaches, maths, ravi).
6 teacher(teaches, science, priya).
7 teacher(teaches, english, kumar).
8
9 student_subject(magdelin_stefey, maths).
10 student_subject(magdelin_stefey, science).
11 student_subject(magdelin_stefey, english).
12
13 relation(Student, Subject, Teacher) :-
14   student_subject(Student, Subject),
15   teacher(teaches, Subject, Teacher).

```

The right pane shows the output of the program:

```

?- student.
Name:- Magdelin Stefey
Register Number:- 192511329
true

?- relation(magdelin_stefey, maths, Teacher).
Teacher = ravi
true

?- relation(magdelin_stefey, science, Teacher).
Teacher = priya
true

?- relation(magdelin_stefey, english, Teacher).
Teacher = kumar
true

?- relation(magdelin_stefey, science, Teacher).

```

At the bottom of the right pane, there are buttons for 'Examples', 'History', 'Solutions', a checkbox for 'table results', and a 'Run' button.

9. Family tree – all relation

Code:

student :-

```

write('Name:- Magdelin Stefey'), nl,
write('Register Number:- 192511329'), nl.

```

```

parent(ravi, Magdelin Stefey).
parent(priya, Magdelin Stefey).
parent(kumar, ravi).
parent(latha, ravi).

```

father(X,Y) :-

parent(X,Y),

X = ravi.

mother(X,Y) :-

parent(X,Y),

X = priya.

child(X,Y) :-

parent(Y,X).

grandparent(X,Y) :-

parent(X,Z),

parent(Z,Y).

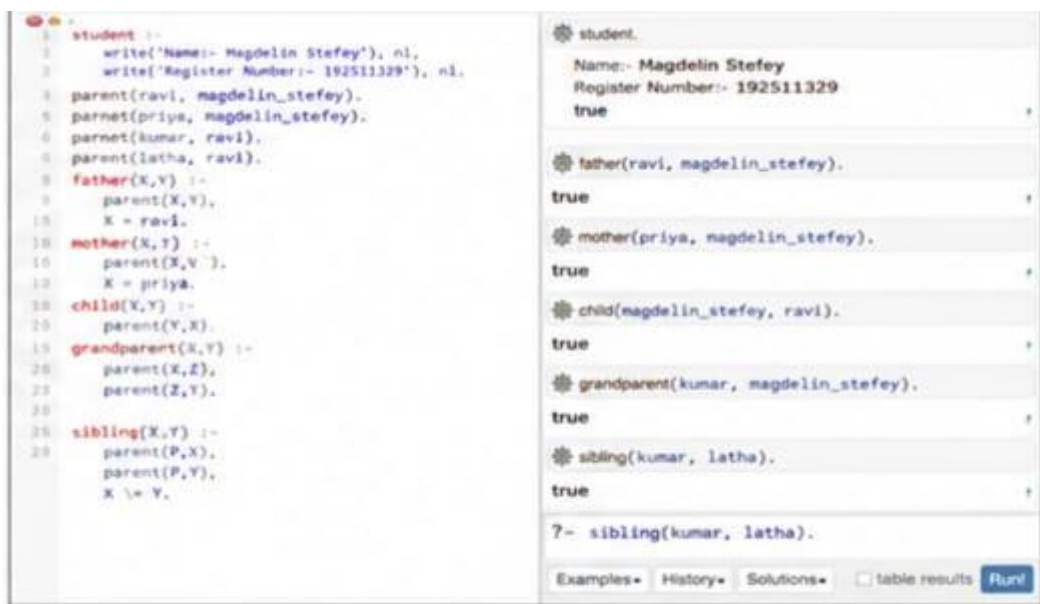
sibling(X,Y) :-

parent(P,X),

parent(P,Y),

X \= Y.

Output:



The screenshot shows a Prolog IDE with a code editor on the left and a results window on the right. The code in the editor defines a student and various family relationships. The results window shows the execution of these queries, all of which return 'true'.

```
1 student :-
2   write('Name:- Magdelin Stefey'), nl,
3   write('Register Number:- 192511329'), nl.
4 parent(ravi, magdelin_stefey).
5 parent(priya, magdelin_stefey).
6 parent(kumar, ravi).
7 parent(latha, ravi).
8 father(X,Y) :-
9   parent(X,Y),
10  X = ravi.
11 mother(X,Y) :-
12   parent(X,Y),
13   X = priya.
14 child(X,Y) :-
15   parent(Y,X).
16 grandparent(X,Y) :-
17   parent(X,Z),
18   parent(Z,Y).
19 sibling(X,Y) :-
20   parent(P,X),
21   parent(P,Y),
22   X \= Y.
```

student.
Name:- Magdelin Stefey
Register Number:- 192511329
true

father(ravi, magdelin_stefey).
true

mother(priya, magdelin_stefey).
true

child(magdelin_stefey, ravi).
true

grandparent(kumar, magdelin_stefey).
true

sibling(kumar, latha).
true

?- sibling(kumar, latha).

Examples History Solutions table results Run!

10. Dieting system based disease

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,  
write('Register Number:- 192511329'), nl.
```

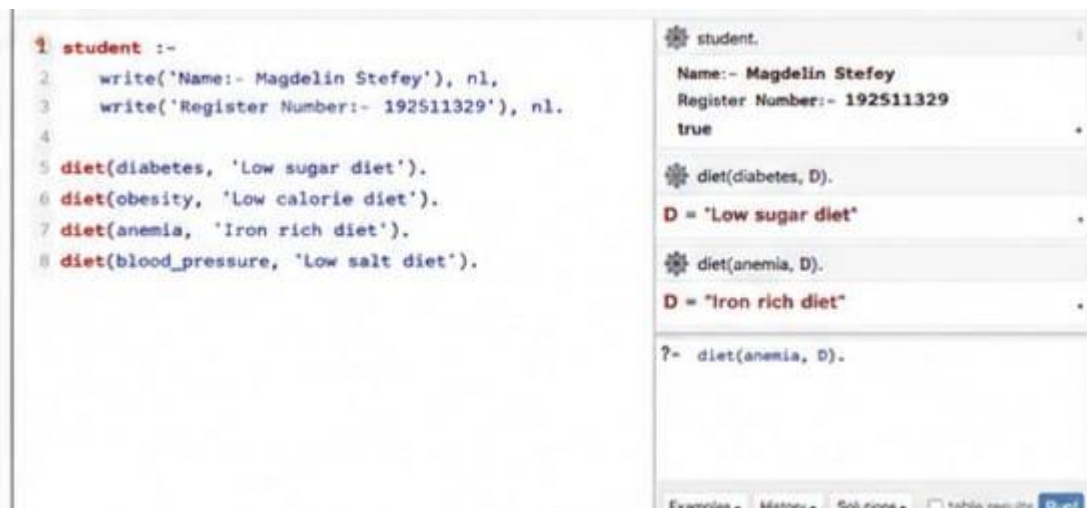
```
diet(diabetes, 'Low sugar diet').
```

```
diet(obesity, 'Low calorie diet').
```

```
diet(anemia, 'Iron rich diet').
```

```
diet(blood_pressure, 'Low salt diet').
```

Output:



The screenshot shows a Prolog IDE with a code editor on the left and an output window on the right. The code editor contains the following Prolog code:

```
1 student :-  
2   write('Name:- Magdelin Stefey'), nl,  
3   write('Register Number:- 192511329'), nl.  
4  
5 diet(diabetes, 'Low sugar diet').  
6 diet(obesity, 'Low calorie diet').  
7 diet(anemia, 'Iron rich diet').  
8 diet(blood_pressure, 'Low salt diet').
```

The output window shows the results of the execution:

```
student.  
Name:- Magdelin Stefey  
Register Number:- 192511329  
true  
  
diet(diabetes, D).  
D = 'Low sugar diet'  
  
diet(anemia, D).  
D = 'Iron rich diet'  
  
?- diet(anemia, D).
```

At the bottom of the output window, there are tabs for 'Examples', 'History', 'Solutions', and a checkbox for 'table results'. A 'Run' button is also visible.

11. Forward / backward chaining

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,  
write('Register Number:- 192511329'), nl.
```

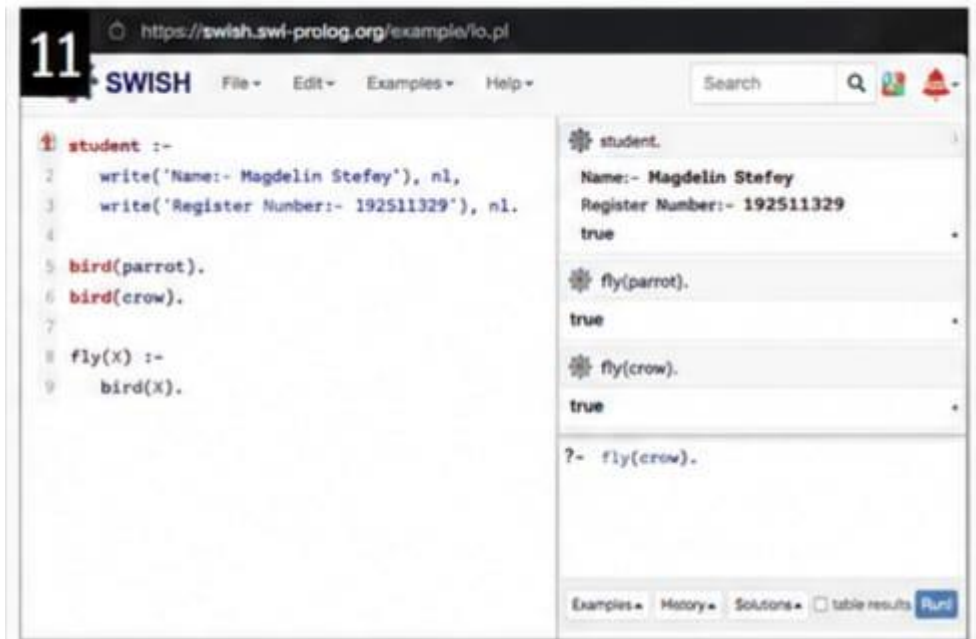
```
bird(parrot).
```

bird(crow).

fly(X) :-

bird(X).

Output:



12. Monkey banana problem

Code:

student :-

write('Name:- Magdelin Stefey'), nl,

write('Register Number:- 192511329'), nl.

monkey(at_door).

banana(on_table).

can_get_banana :-

monkey(at_door),

banana(on_table),

write('Monkey moves to table and gets banana').

Output:



The screenshot shows a Prolog IDE with a code editor on the left and an output window on the right. The code in the editor is as follows:

```
1 student :-  
2   write('Name:- Magdelin Stefey'), nl,  
3   write('Register Number:- 192511329'), nl.  
4  
5 monkey(at_door).  
6 banana(on_table).  
7  
8 can_get_banana :-  
9   monkey(at_door),  
10  banana(on_table),  
11  write('Monkey moves to table and gets banana').
```

The output window on the right displays the results of running the code:

```
student.  
Name:- Magdelin Stefey  
Register Number:- 192511329  
true  
  
can_get_banana.  
Monkey moves to table and gets banana  
true  
  
?- can_get_banana.
```

At the bottom of the output window, there are tabs for 'Examples', 'History', and 'Solutions', along with a checkbox for 'table results' and a 'Run' button.

13. Fruit and color – backtracking

Code:

student :-

write('Name:- Magdelin Stefey'), nl,

write('Register Number:- 192511329'), nl.

fruit(apple, red).

fruit(banana, yellow).

fruit(grapes, green).

fruit(orange, orange).

Output:

```
student :-
2   write('Name:- Magdelin Stefey'), nl,
3   write('Register Number:- 192511329'), nl.
4
5   fruit(apple, red).
6   fruit(banana, yellow).
7   fruit(grapes, green).
8   fruit(orange, orange).
```

student.

Name:- Magdelin Stefey
Register Number:- 192511329
true

fruit(X, red).
X = apple

fruit(X, yellow).
X = banana

?- fruit(X, yellow).

Examples History Solutions ☐ table results Run

14. Bird fly or not

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,
write('Register Number:- 192511329'), nl.
```

bird(parrot).

bird(crow).

bird(penguin).

can_fly(parrot).

can_fly(crow).

cannot_fly(penguin).

Output:

```
1 student :-
2   write('Name:- Magdelin Stefey'), nl,
3   write('Register Number:- 192511329'), nl.
4
5 bird(parrot).
6 bird(crow).
7 bird(penguin).
8
9 can_fly(parrot).
10 can_fly(crow).
11 cannot_fly(penguin).
```

student.

Name:- Magdelin Stefey
Register Number:- 192511329
true

can_fly(parrot).
true

cannot_fly(penguin).
true

?= cannot_fly(penguin).

Examples History Solutions ☐ table results [Print](#)

15. Grade find

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,
write('Register Number:- 192511329'), nl.
```

grade(Mark, 'A') :-

```
Mark >= 90, !.
```

grade(Mark, 'B') :-

```
Mark >= 75,
Mark < 90, !.
```

grade(Mark, 'C') :-

```
Mark >= 60,
Mark < 75, !.
```

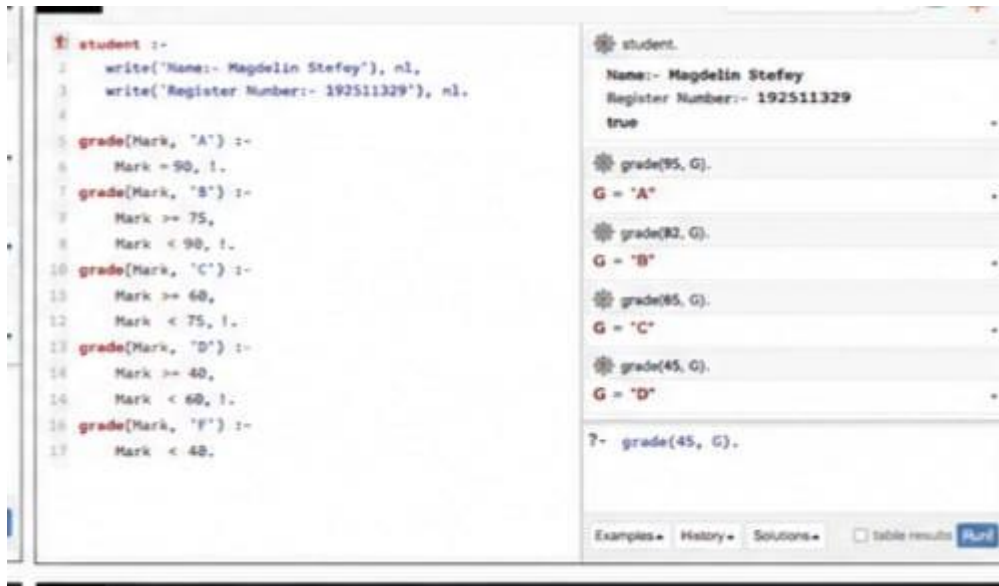
grade(Mark, 'D') :-

```
Mark >= 40,
Mark < 60, !.
```

grade(Mark, 'F') :-

Mark < 40.

Output:



The screenshot shows a Prolog IDE with a code editor on the left and an output window on the right. The code in the editor defines a `student` predicate and a `grade` predicate. The output window shows the execution of these predicates, displaying the student's name, register number, and the resulting grade for a given mark.

```
student :-  
    write('Name:- Magdelin Stefey'), nl,  
    write('Register Number:- 192511329'), nl.  
  
grade(Mark, 'A') :-  
    Mark >= 90, !.  
grade(Mark, 'B') :-  
    Mark >= 75,  
    Mark < 90, !.  
grade(Mark, 'C') :-  
    Mark >= 60,  
    Mark < 75, !.  
grade(Mark, 'D') :-  
    Mark >= 40,  
    Mark < 60, !.  
grade(Mark, 'F') :-  
    Mark < 40.
```

Output:

```
student.  
Name:- Magdelin Stefey  
Register Number:- 192511329  
true  
  
grade(95, G).  
G = 'A'  
  
grade(82, G).  
G = 'B'  
  
grade(85, G).  
G = 'C'  
  
grade(45, G).  
G = 'D'  
  
?- grade(45, G).
```

16. EB bill Program

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,  
write('Register Number:- 192511329').
```

bill(U, B) :-

U <= 100, B is 0, !.

bill(U, B) :-

U <= 200, B is (U-100)*2.25, !.

bill(U, B) :-

U <= 500, B is 225+(U-200)*4.5, !.

bill(U, B) :-

B is 1575+(U-500)*6.

Output:

```

1 student :-
2   write('Name:- Magdelin Stefey'), nl,
3   write('Register Number:- 192511329'), nl.
4
5 bill(U, B) :-
6   U <= 100, B is U, !.
7
8 bill(U, B) :-
9   U <= 200, B is (U-100)*2.25, !.
10
11 bill(U, B) :-
12   U <= 500, B is 225+(U-200)*4.5, !.
13
14 bill(U, B) :-
15   B is 1575+(U-500)*6.
16

```

student.

Name:- Magdelin Stefey
Register Number:- 192511329
true

bill(300, B).
X = 675.0

bill(150, B).
B = 112.5

bill(100, B).
B = 100

bill(200, B).
B = 225.0

?- bill(200, B).

Examples History Solutions ☐ table results Run

17. Student database

Code:

student :-

```

write('Name:- Magdelin Stefey'), nl,
write('Register Number:- 192511329'), nl.

```

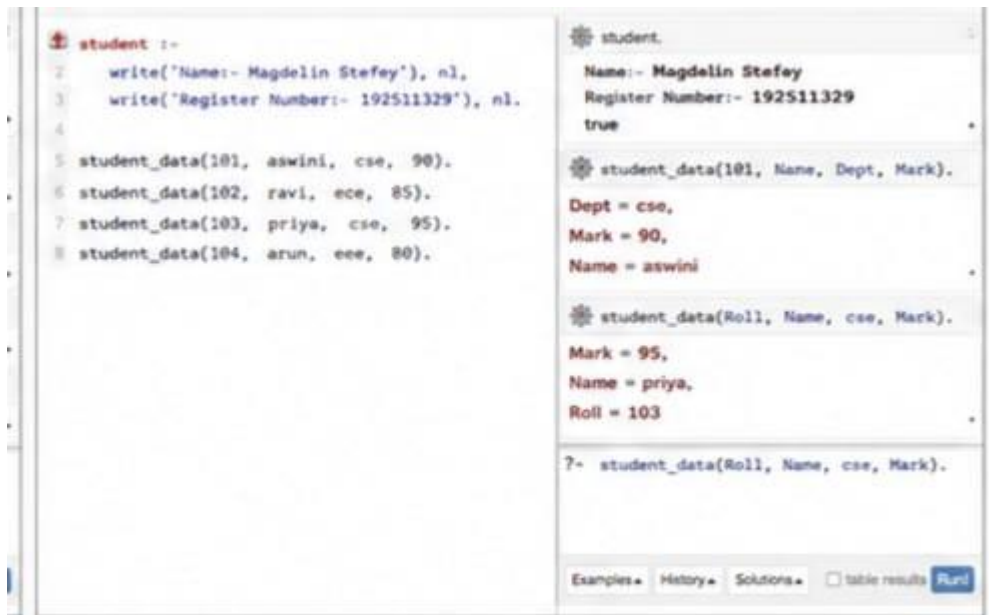
student_data(101, Magdelin Stefey, ece, 90).

student_data(102, ravi, ece, 85).

student_data(103, priya, cse, 95).

student_data(104, arun, eee, 80).

Output:



```
student :-
1 write('Name:- Magdelin Stefey'), nl,
2 write('Register Number:- 192511329'), nl,
3
4
5 student_data(101, aswini, cse, 90).
6 student_data(102, ravi, ece, 85).
7 student_data(103, priya, cse, 95).
8 student_data(104, arun, eee, 80).
```

```
student.
Name:- Magdelin Stefey
Register Number:- 192511329
true

student_data(101, Name, Dept, Mark).
Dept = cse,
Mark = 90,
Name = aswini

student_data(Roll, Name, cse, Mark).
Mark = 95,
Name = priya,
Roll = 103

?- student_data(Roll, Name, cse, Mark).
```

18. Print 1 to 10

Code:

student :-

```
write('Name:- Magdelin Stefey'), nl,
write('Register Number:- 192511329'), nl.
```

print_numbers(10).

print_numbers(N) :-

N > 10, !.

print_numbers(N) :-

```
write(N), nl,
N1 is N + 1,
print_numbers(N1).
```

Output:

```

1 student :-
2   write('Name:- Magdelin Stefey'), nl,
3   write('Register Number:- 192511329'), nl.
4
5 print_numbers(10).
6
7 print_numbers(N) :-
8   N > 10, !,
9   write(N), nl,
10  N1 is N - 1,
11  print_numbers(N1).

```

```

student.
  Name:- Magdelin Stefey
  Register Number:- 192511329
  true

print_numbers(1).
  1
  2
  3
  4
  5
  6
  7
  8
  9
  10
  true
  true

?- print_numbers(1).

```

19. N queens Problem

Code:

student :-

```

write('Name:- Magdelin Stefey'), nl,
write('Register Number:- 192511329'), nl.

```

queens(S) :-

```

permutation([1,2,3,4], S),
safe(S).

```

safe([]).

safe([X|Xs]) :-

```

safe(Xs),
check(X, Xs, 1).

```

check(_, [], _).

check(X, [Y|Ys], D) :-

X =\= Y,

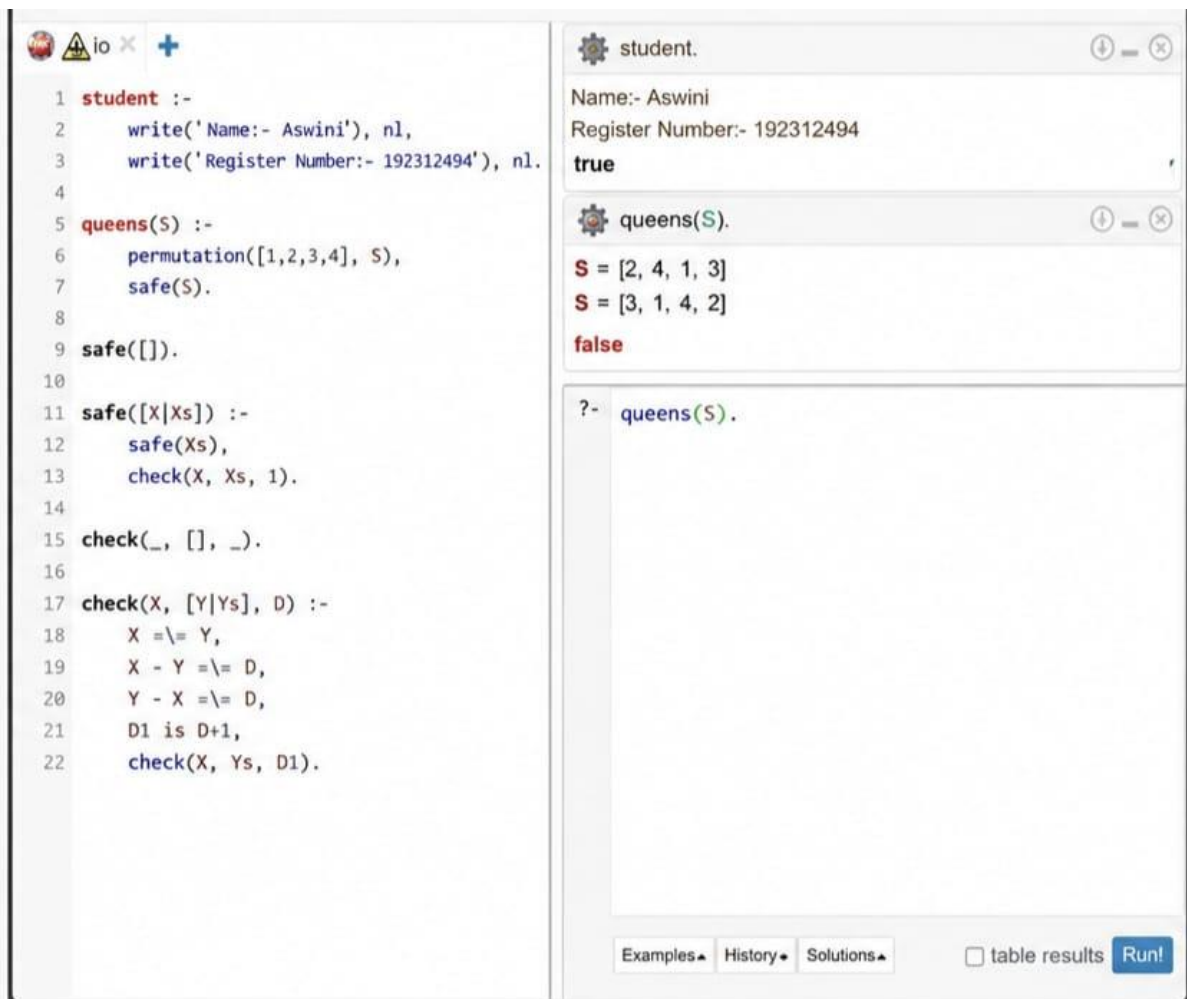
X-Y =\= D,

Y-X =\= D,

D1 is D+1,

check(X, Ys, D1).

Output:



The screenshot shows a Prolog IDE with a code editor on the left and a console on the right. The code editor contains the following Prolog code:

```
1 student :-
2     write('Name:- Aswini'), nl,
3     write('Register Number:- 192312494'), nl.
4
5 queens(S) :-
6     permutation([1,2,3,4], S),
7     safe(S).
8
9 safe([]).
10
11 safe([X|Xs]) :-
12     safe(Xs),
13     check(X, Xs, 1).
14
15 check(_, [], _).
16
17 check(X, [Y|Ys], D) :-
18     X =\= Y,
19     X - Y =\= D,
20     Y - X =\= D,
21     D1 is D+1,
22     check(X, Ys, D1).
```

The console on the right shows the output of the code. It has two tabs: 'student.' and 'queens(S)'. The 'student.' tab shows the output: 'Name:- Aswini' and 'Register Number:- 192312494', followed by 'true'. The 'queens(S)' tab shows the output: 'S = [2, 4, 1, 3]' and 'S = [3, 1, 4, 2]', followed by 'false'. At the bottom of the console, there is a prompt '?- queens(S).' and a 'Run!' button.

20. Reverse a number

Code:

student :-

write('Name:- Magdelin Stefey'), nl,

write('Register Number:- 192511329'), nl.

reverse_num(N, R) :-

reverse_num(N, 0, R).

reverse_num(0, R, R).

reverse_num(N, A, R) :-

D is N mod 10,

N1 is N // 10,

A1 is A * 10 + D,

reverse_num(N1, A1, R).

Output:

The screenshot shows the SWISH Prolog IDE interface. The left pane contains the Prolog code, and the right pane shows the execution results for two queries.

Code in the left pane:

```
1 student :-
2   write('Name:- Aswini'), nl,
3   write('Register Number:- 192312494'), nl.
4
5 reverse_num(N, R) :-
6   reverse_num(0, N, R).
7
8 reverse_num(D, R, R).
9
10 reverse_num(N, A, R) :-
11   D is N mod 10,
12   N1 is N // 10,
13   A1 is A * 10 + D,
14   reverse_num(N1, A1, R).
```

Execution results in the right pane:

student.

Name:- Aswini
Register Number:- 192312494
true

reverse_num(1234,R).

R = 4321
R = 43210
R = 432100
R = 4321000
R = 43210000
R = 432100000
R = 4321000000

Next 10 100 1,000 Stop

reverse_num(123,R).

R = 321
R = 3210
R = 32100
R = 321000
R = 3210000
R = 32100000

Next 10 100 1,000 Stop

?- reverse_num(123, R).

Examples History Solutions ☐ table results Run!